

CS5707 Artificial Intelligence
Search

Yongmin Li

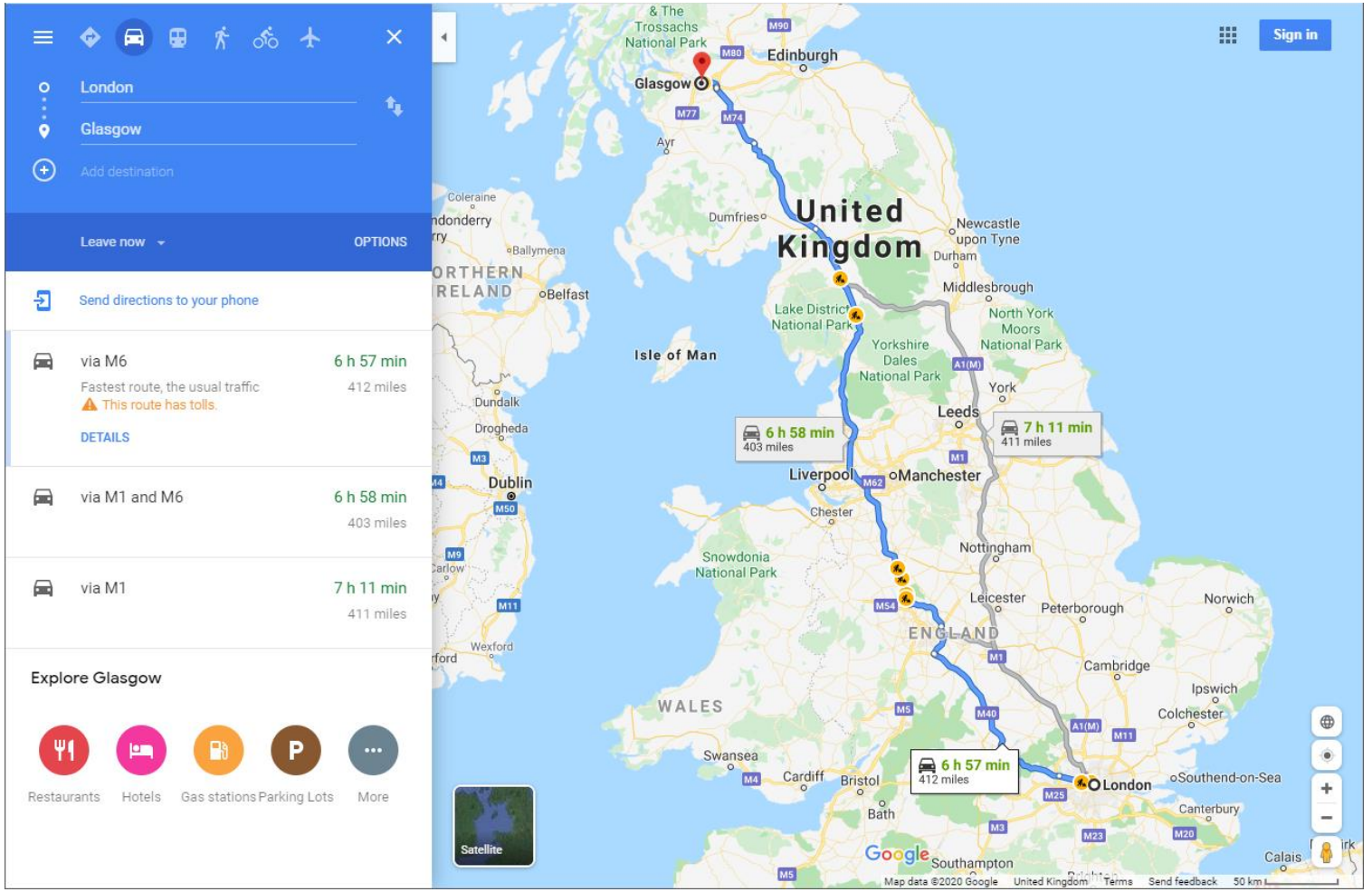
Overview

- **Breadth-first search**
- **Depth-first search**
- **Uniform-cost search**
- **Greedy best-first search**
- **A* search**

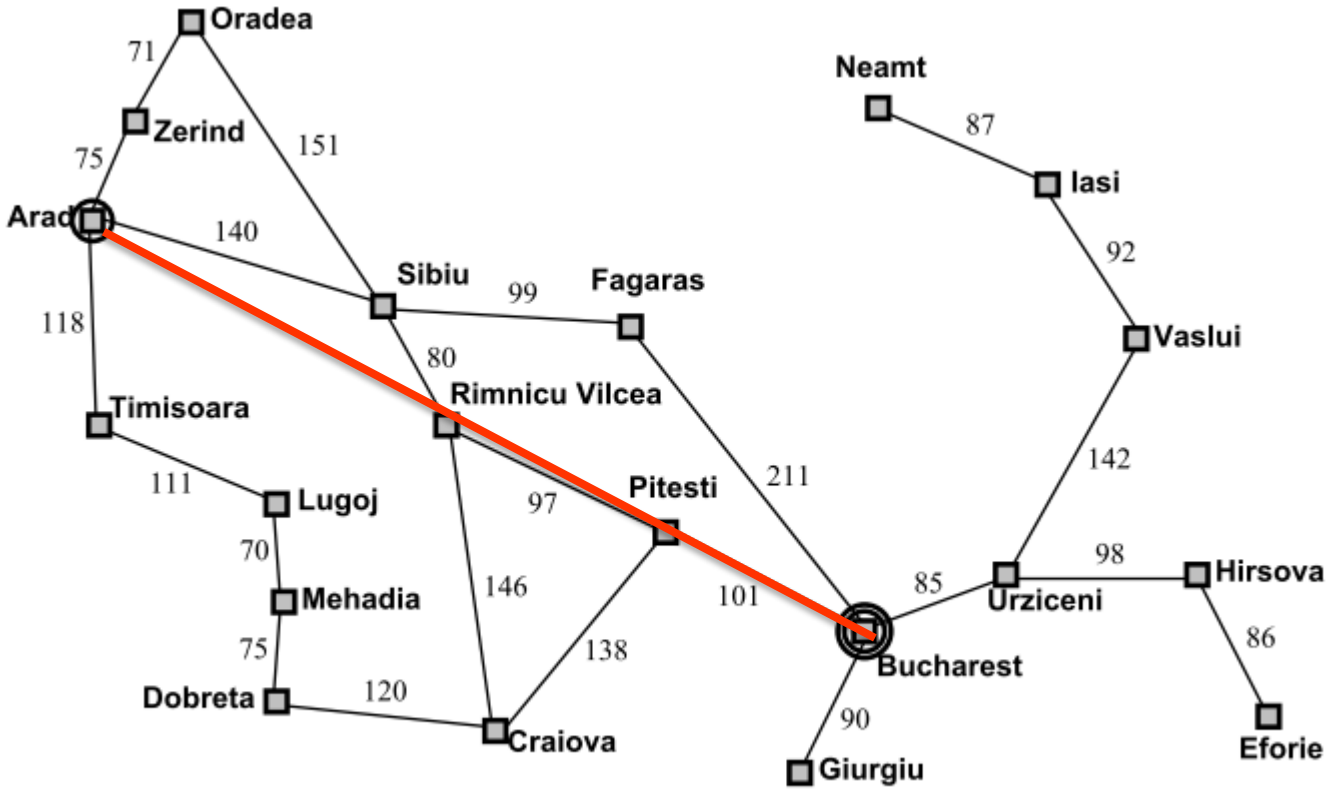
Reading

[1] Russell, S. and Norvig, P. (2021). §3, Solving problems by searching.

Google Maps


London
Glasgow
Add destination
Leave now
OPTIONS
Send directions to your phone
via M6
6 h 57 min
Fastest route, the usual traffic
412 miles
This route has tolls.
DETAILS
via M1 and M6
6 h 58 min
403 miles
via M1
7 h 11 min
411 miles
Explore Glasgow
Restaurants
Hotels
Gas stations
Parking Lots
More
Sign in
United Kingdom
Edinburgh
Glasgow
Belfast
Dublin
Isle of Man
Liverpool
Manchester
Leeds
Nottingham
Leicester
Peterborough
Norwich
Cambridge
Ipswich
Colchester
London
Southend-on-Sea
Canterbury
Calais
Bristol
Cardiff
Swansea
Bath
Southampton
Newcastle upon Tyne
Middlesbrough
Durham
York
Chester
Dumfries
Ayr
Lake District National Park
Yorkshire Dales National Park
North York Moors National Park
Snowdonia National Park
ENGLAND
WALES
6 h 58 min
403 miles
7 h 11 min
411 miles
6 h 57 min
412 miles
Satellite
Google
Map data ©2020 Google
United Kingdom
Terms
Send feedback
50 km

An Example



Romania: Travel from Arad to Bucharest

Notations

A search problem can be defined by 4 items:

- **state**
- **action**
- **goal test**
- **path cost**

Notations

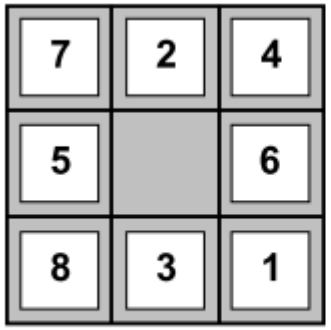
For the Romania problem:

- **state: “at Arad”**
- **action: “Arad → Zerind”**
- **goal test: “at Bucharest” (destination reached?)**
- **path cost: sum of distances**

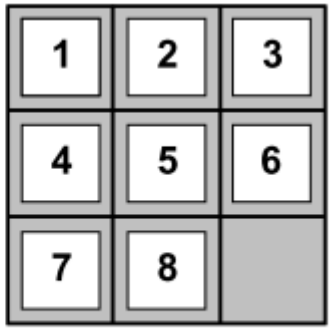
Question?

How do you define the 8-puzzle problem?

- **state?**
- **action?**
- **goal test?**
- **path cost?**



Start State



Goal State

Uninformed Search

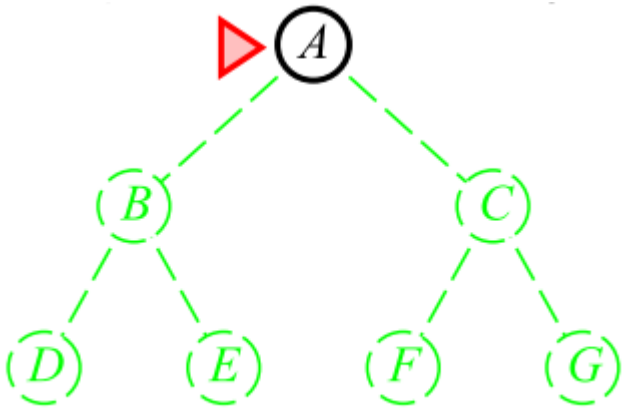
- **Breadth-first search**
- **Depth-first search**
- **Uniform-cost search**

Breadth-First Search

- **Expand the shallowest unexpended node**
- **Use an FIFO queue for the frontier**

Breadth-First Search

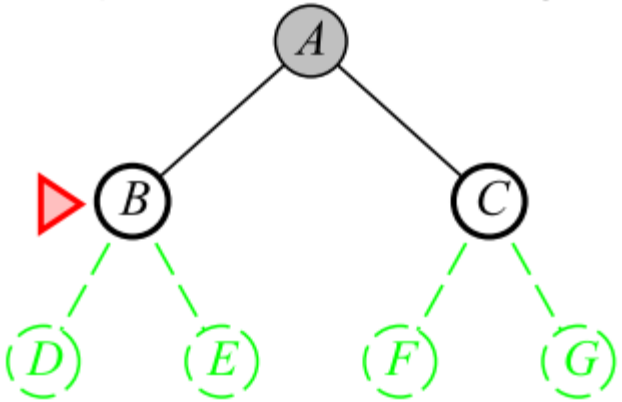
Queue management: alphabetical order



Explored: {A}
Queue: {BC}

Breadth-First Search

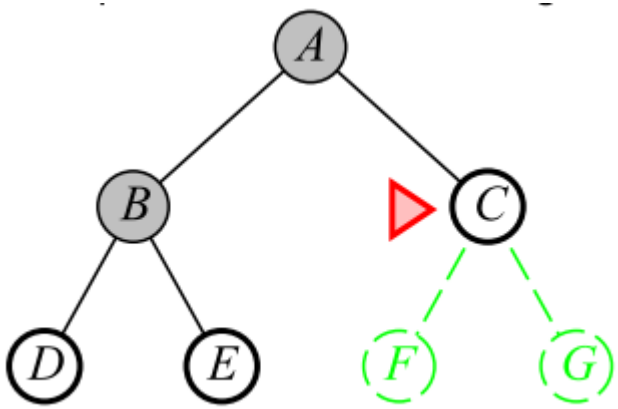
Queue management: alphabetical order



Explored: {AB}
Queue: {CDE}

Breadth-First Search

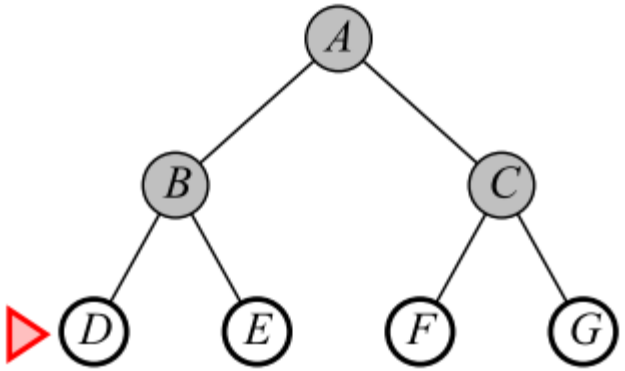
Queue management: alphabetical order



Explored: {ABC}
Queue: {DEFG}

Breath-First Search

Queue management: alphabetical order



Explored: {ABCD}
Queue: {EFG}

Breadth-First Search

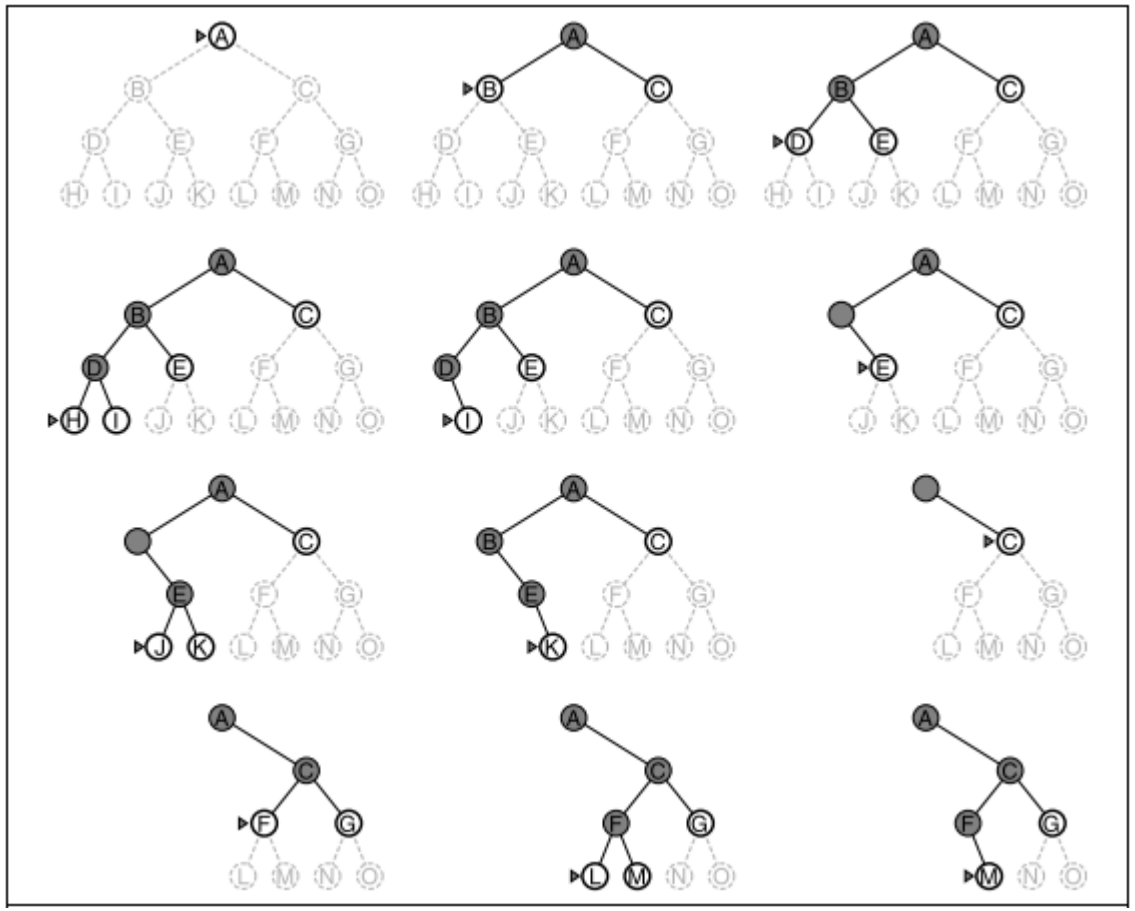
```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)  /* chooses the shallowest node in frontier */
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
        frontier ← INSERT(child, frontier)
```

Depth-First Search

- **Expand the deepest unexpended node**
- **Use an LIFO queue for the frontier**

Depth-First Search

Queue management: reverse alphabetical order



Explored: {A}
Queue: {CB}

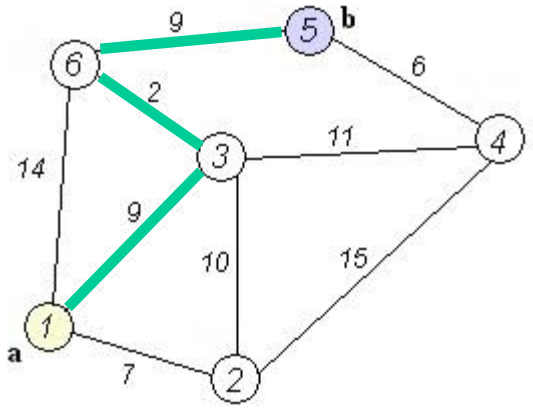
Explored: {AB}
Queue: {CED}

Explored: {ABD}
Queue: {CEIH}

…

Uniform-Cost Search

- **BFS is optimal for equal step cost**
- **Extension to any step costs**



Example:

Start: ①

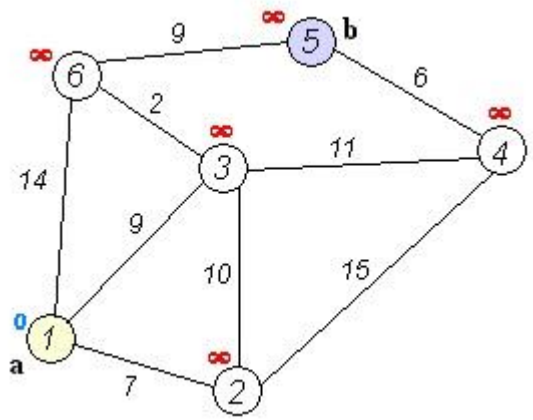
Target: ⑤

S: Explored

Q: Queue

Uniform-Cost Search

Round	1	2	3	4	5	6
0	0	∞	∞	∞	∞	∞

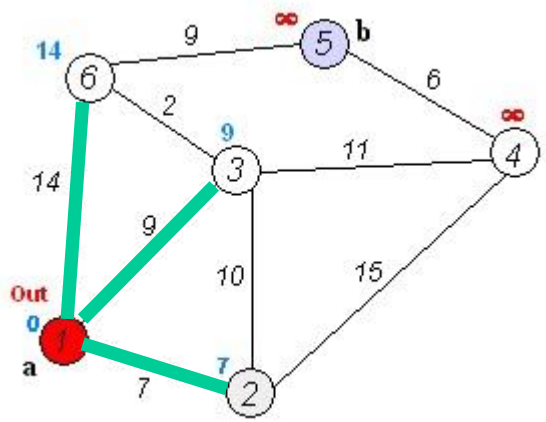


S: {}

Q: {123456}

Uniform-Cost Search

Round	1	2	3	4	5	6
0	0	∞	∞	∞	∞	∞
1		7	9			14

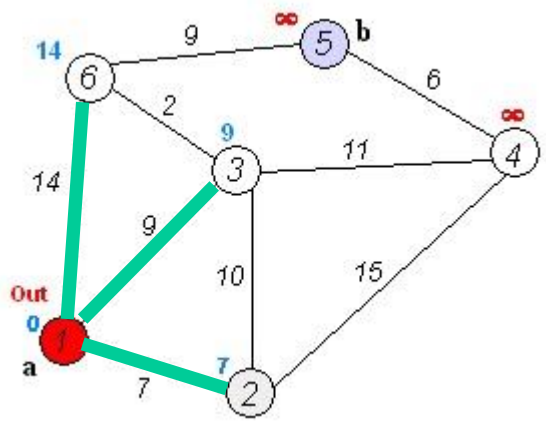


S: {1}

Q: {23456}

Uniform-Cost Search

Round	1	2	3	4	5	6
0	0	∞	∞	∞	∞	∞
1		7	9	∞	∞	14

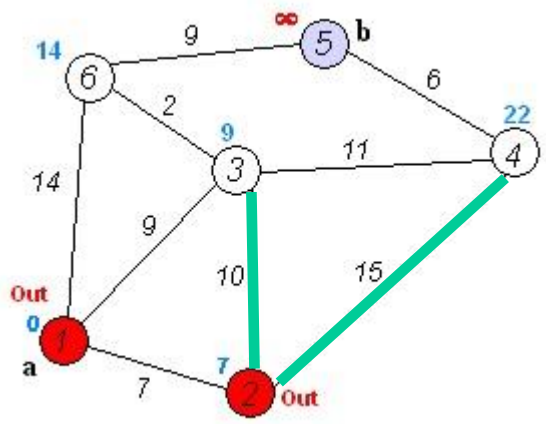


S: {1}

Q: {23456}

Uniform-Cost Search

Round	1	2	3	4	5	6
0	0	∞	∞	∞	∞	∞
1		7	9	∞	∞	14
2			7+10>9	22		

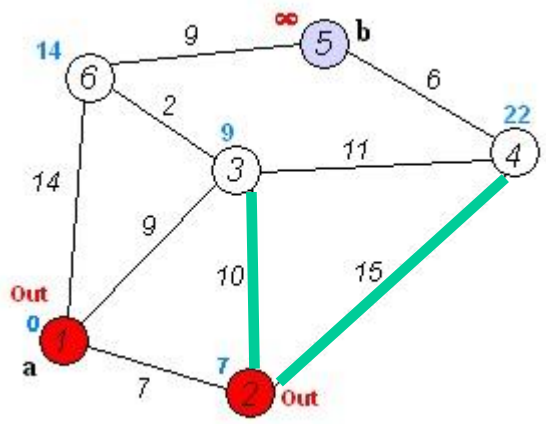


S: {12}

Q: {3456}

Uniform-Cost Search

Round	1	2	3	4	5	6
0	0	∞	∞	∞	∞	∞
1		7	9	∞	∞	14
2			9	22	∞	14

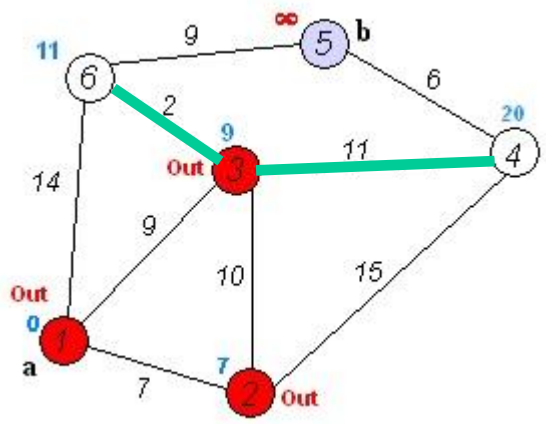


S: {12}

Q: {3456}

Uniform-Cost Search

Round	1	2	3	4	5	6
0	0	∞	∞	∞	∞	∞
1		7	9	∞	∞	14
2			9	22	∞	14
3				9+11<22		9+2<14

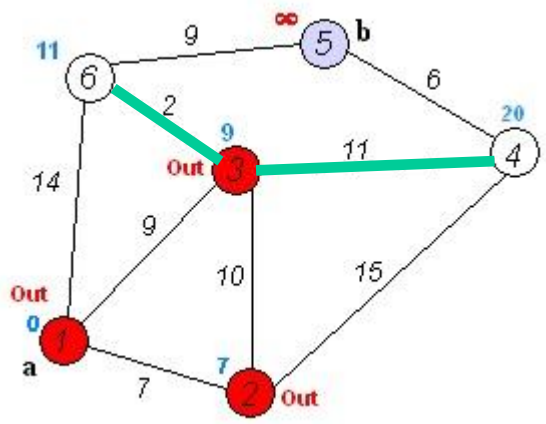


S: {123}

Q: {456}

Uniform-Cost Search

Round	1	2	3	4	5	6
0	0	∞	∞	∞	∞	∞
1		7	9	∞	∞	14
2			9	22	∞	14
3				20	∞	11

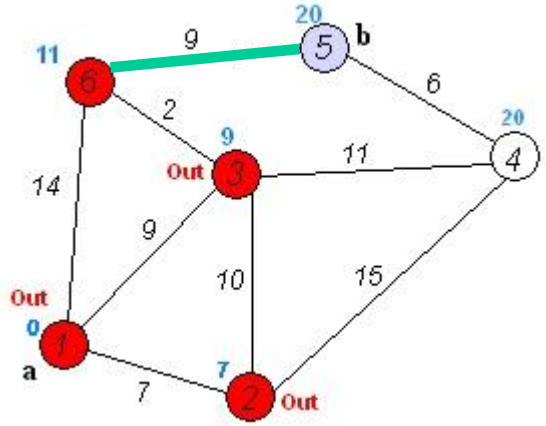


S: {123}

Q: {456}

Uniform-Cost Search

Round	1	2	3	4	5	6
0	0	∞	∞	∞	∞	∞
1		7	9	∞	∞	14
2			9	22	∞	14
3				20	∞	11
4					20	

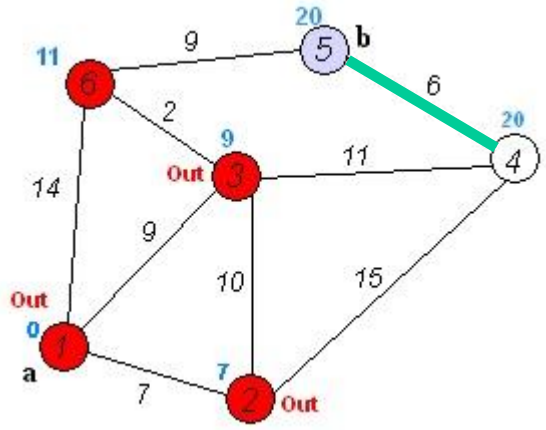


S: {1236}

Q: {45}

Uniform-Cost Search

Round	1	2	3	4	5	6
0	0	∞	∞	∞	∞	∞
1		7	9	∞	∞	14
2			9	22	∞	14
3				20	∞	11
4				20	20	
5						



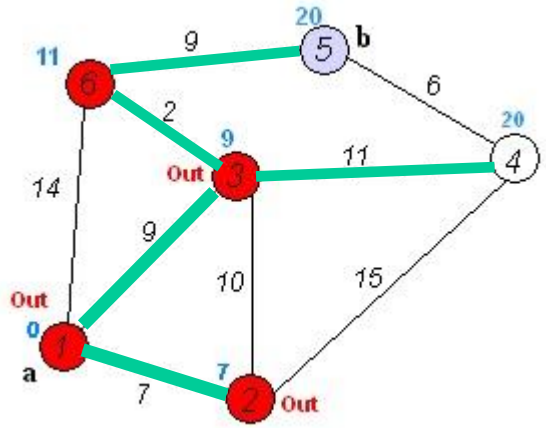
S: {12365}

Q: {4}

What Is the Path?

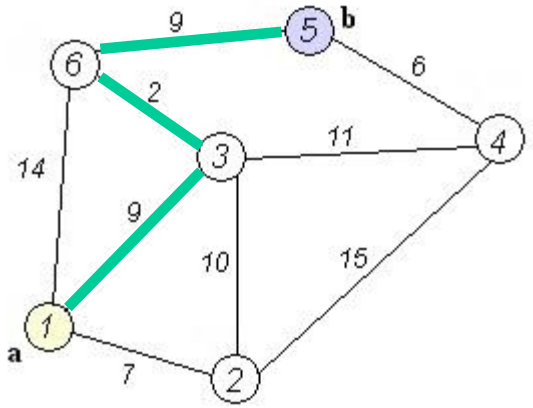
Just keep a record of the previous nodes when updating the distance!

Round	1	2	3	4	5	6
0	0	∞	∞	∞	∞	∞
1		7(1)	9(1)	∞	∞	14(1)
2			9(1)	22(2)	∞	14(1)
3				20(3)	∞	11(3)
4				20(3)	20(6)	
5						



Uniform-Cost Search

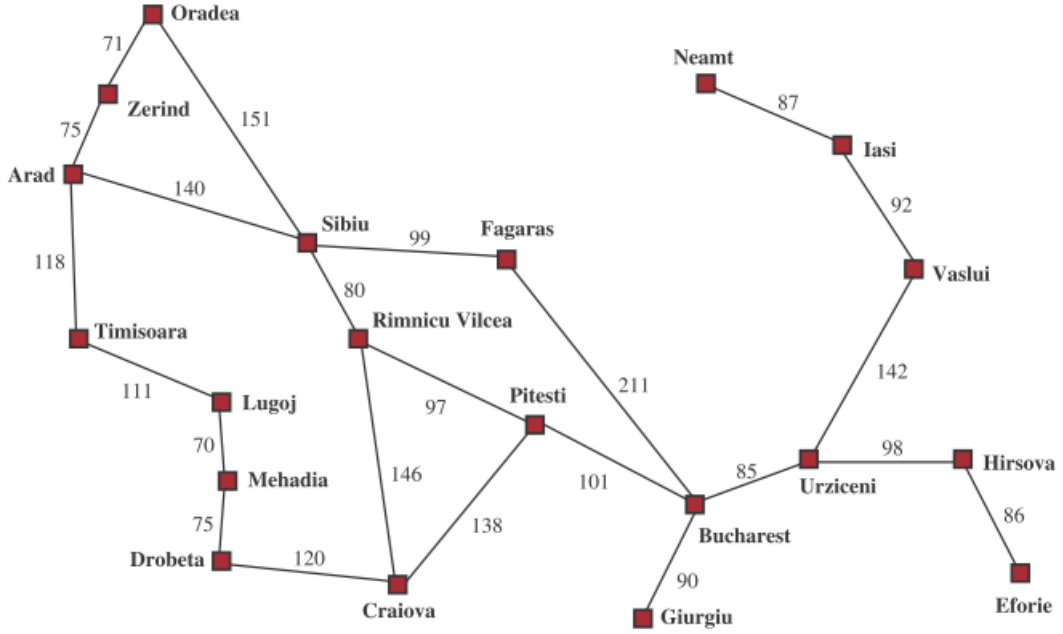
- **Are you familiar with this algorithm?**
- **It is (almost) the Dijkstra Algorithm!**



Informed (Heuristic) Search

- **Greedy best-first search**
- **A* search**
- **Heuristics**

Greedy Best-First Search

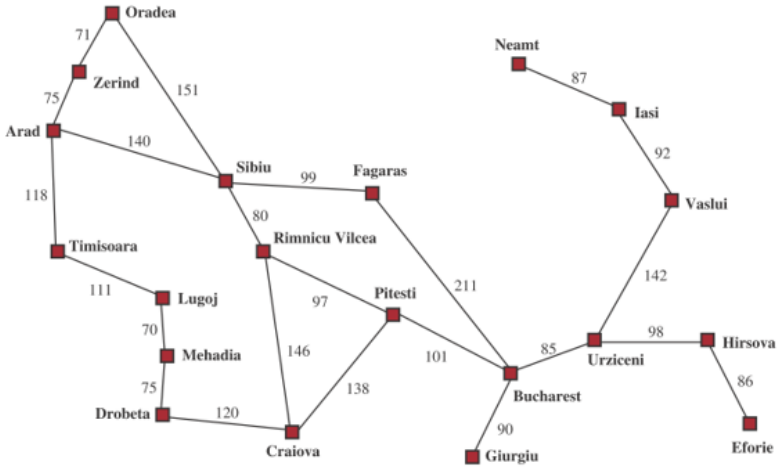


Straight-line distance to Bucharest

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Greedy Best-First Search

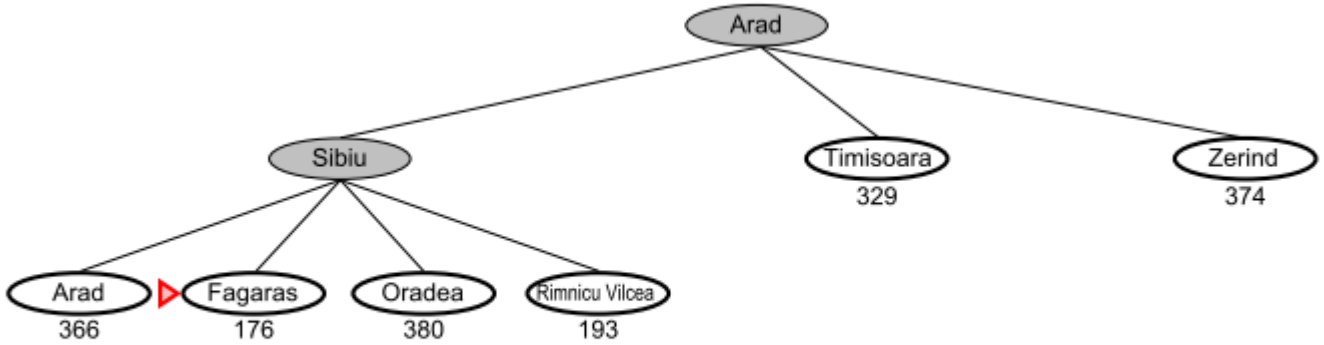


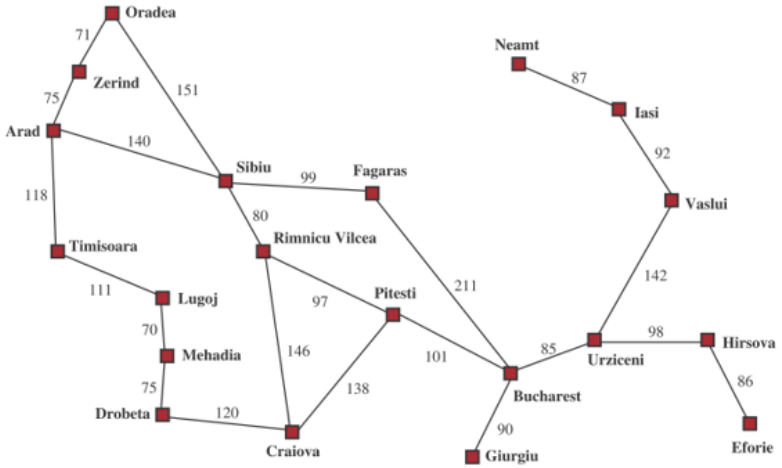


Straight-line distance to Bucharest

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Greedy Best-First Search

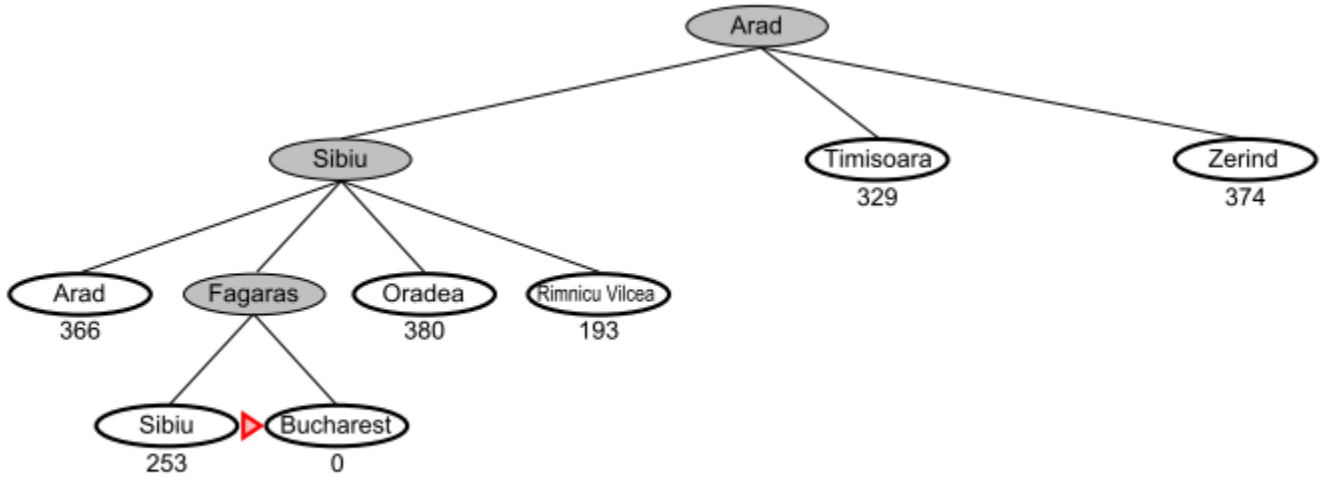


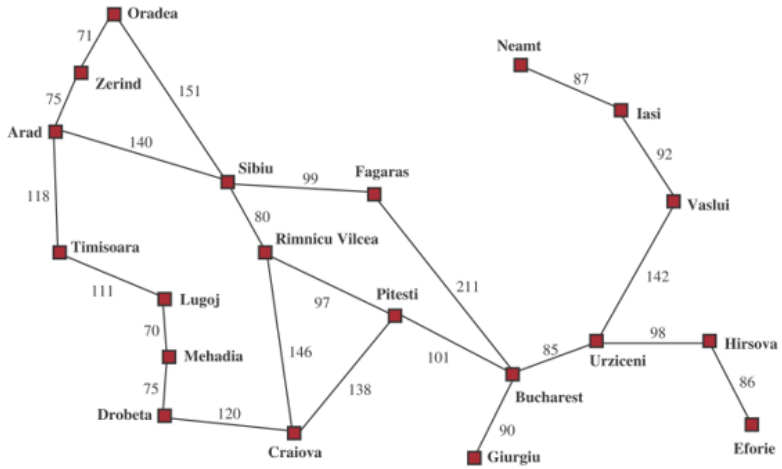


Straight-line distance to Bucharest

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Greedy Best-First Search

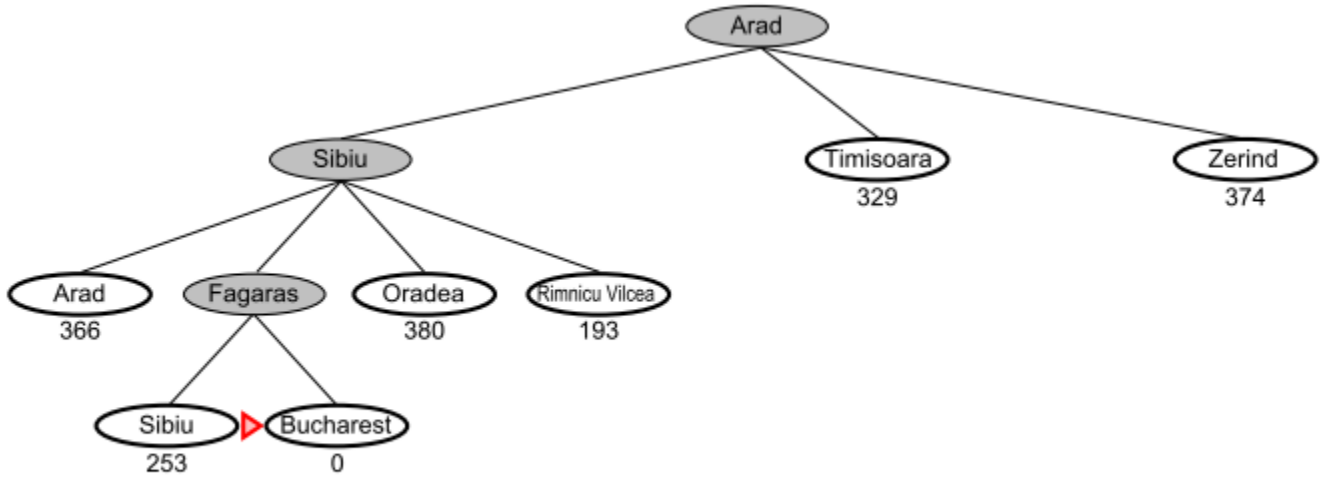




Straight-line distance to Bucharest

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

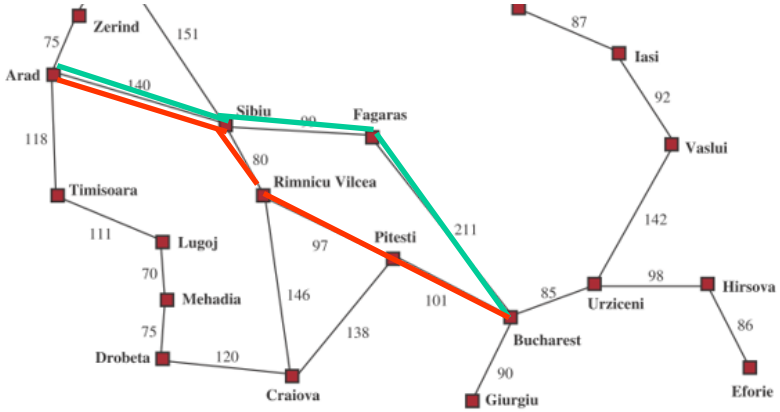
Greedy Best-First Search



But is this the shortest route?

Green route: 450

Red route: 418 !!!



Straight-line distance to Bucharest

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

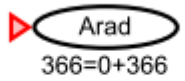
A* Search

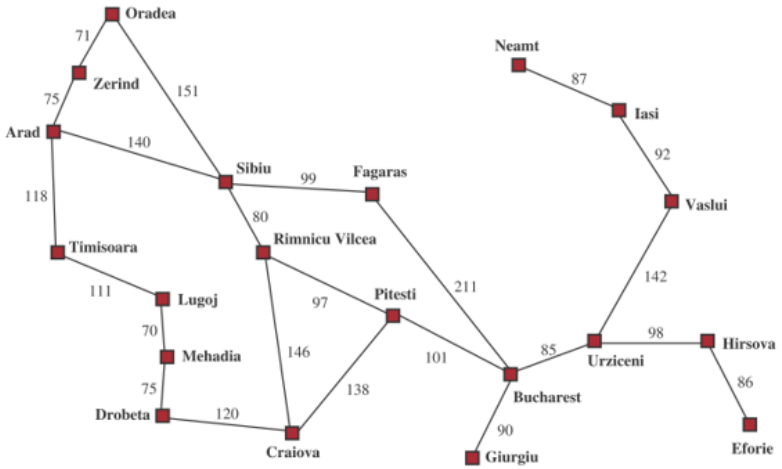
Evaluation Function:

$$f(n) = g(n) + h(n)$$

g(n): cost so far to reach *n*
h(n): estimated cost to goal from *n*
f(n): estimated total cost through *n*

A* Search



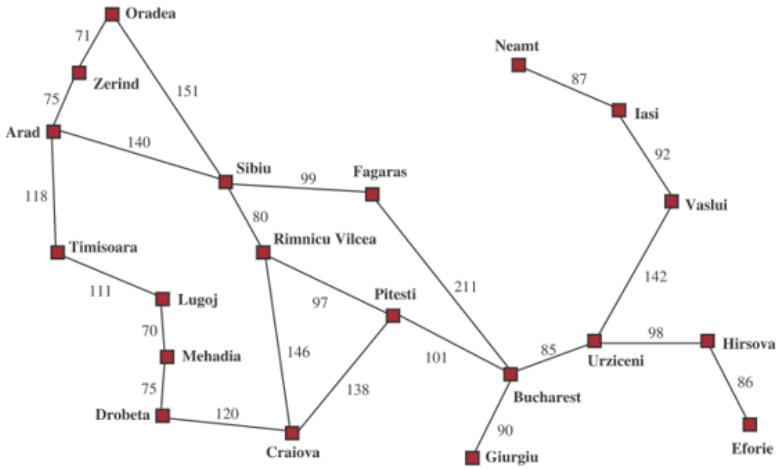


Straight-line distance to Bucharest

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

A* Search

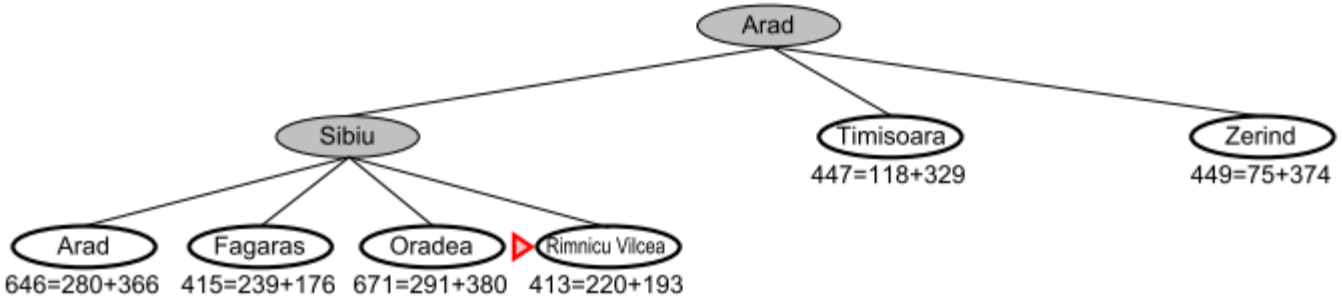


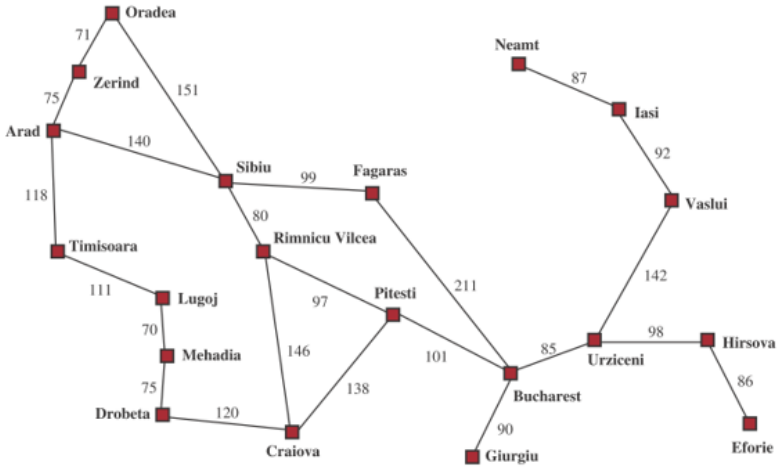


Straight-line distance to Bucharest

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

A* Search

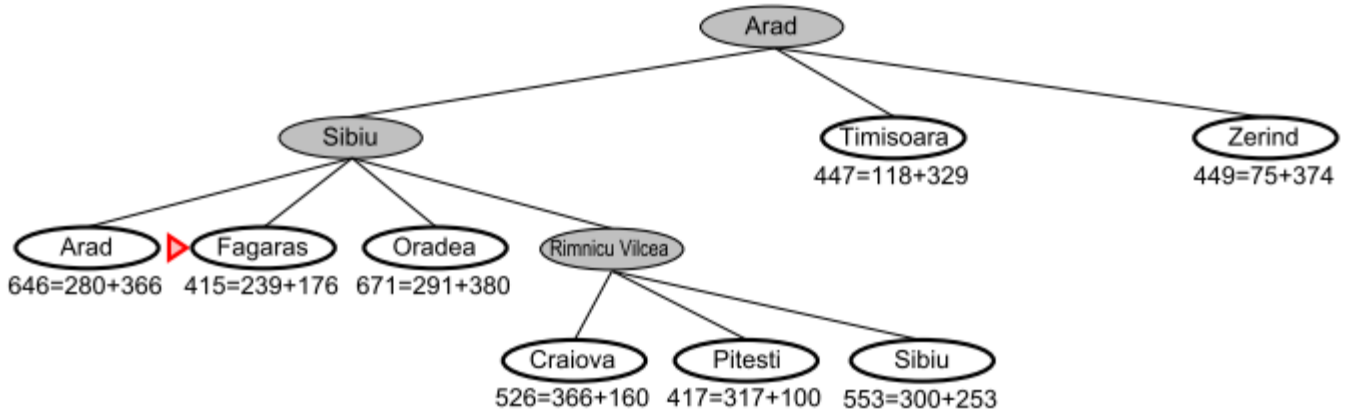


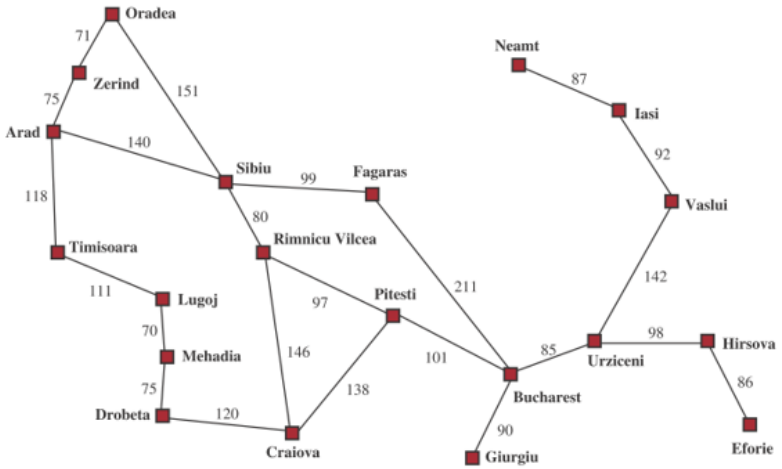


Straight-line distance to Bucharest

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

A* Search

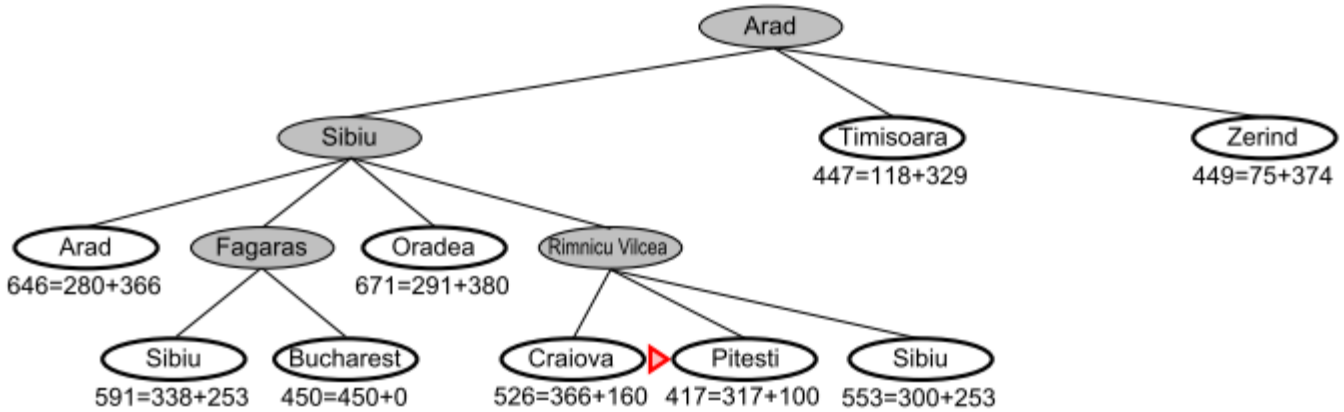


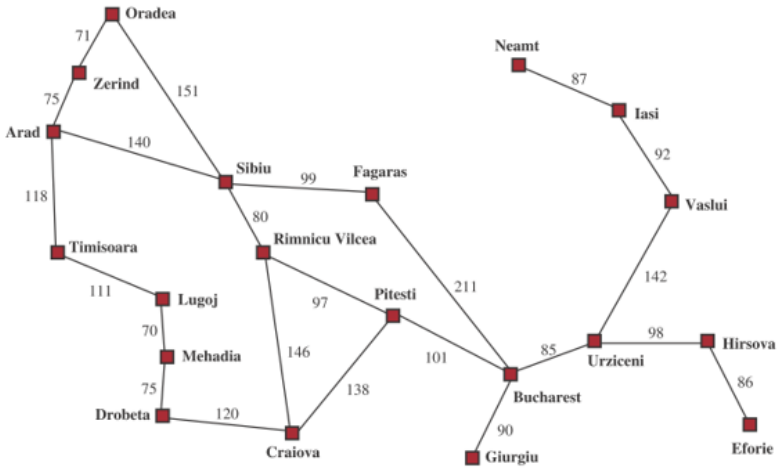


Straight-line distance to Bucharest

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

A* Search

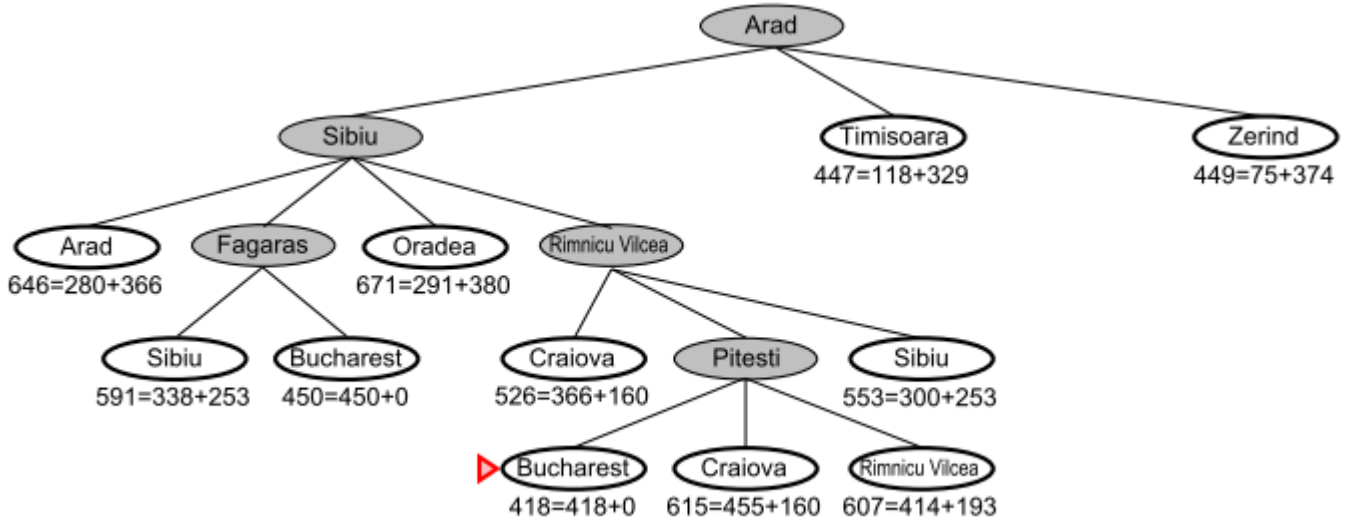


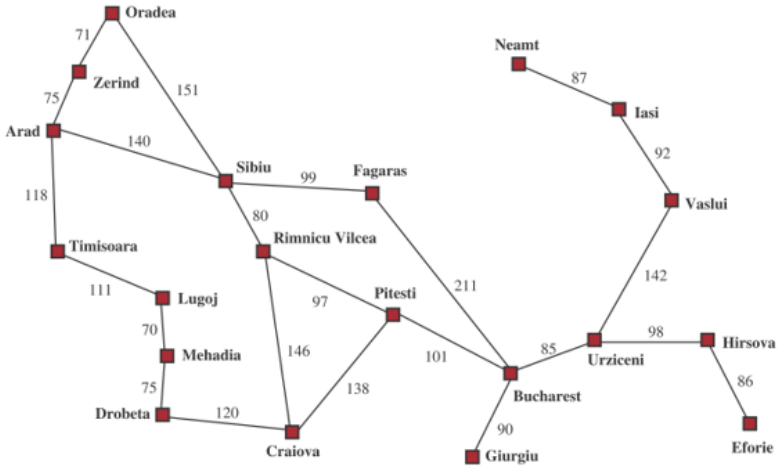


Straight-line distance to Bucharest

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

A* Search





Straight-line distance to Bucharest

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

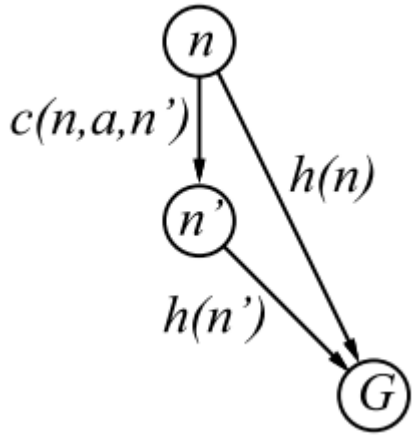
A* Search: Conditions for Optimality

Admissibility: never overestimate the cost

$h(n) <= c(n)$

Consistency: triangle inequality

$|h(n) – h(n')| <= c(n, a, n')$



h(n): estimated cost to goal from *n*

c(n): true cost from *n to goal*

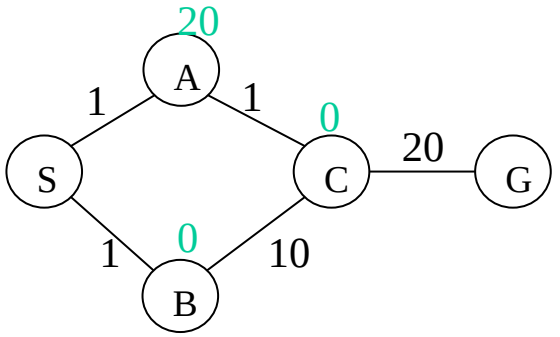
cost(n, a, n'): true cost from *n* to *n'* by action *a*

A* Search:
Conditions for Optimality

Is this heuristic

- admissible?
- consistent?

Can A* search find the optimal route from S to G when first reaching G?



Why?

Conclusion

- **Uninformed search**
  - **Breadth-first search**
  - **Depth-first search**
  - **Uniform-cost search (Dijkstra Algorithm)**
- **Informed (heuristic) search**
  - **Greedy best-first search**
  - **A* search**